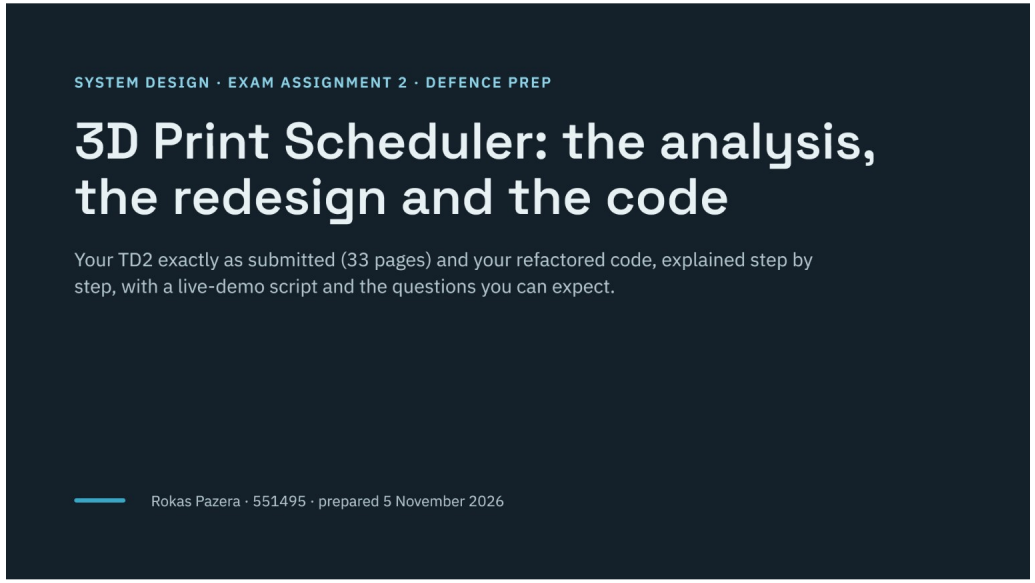
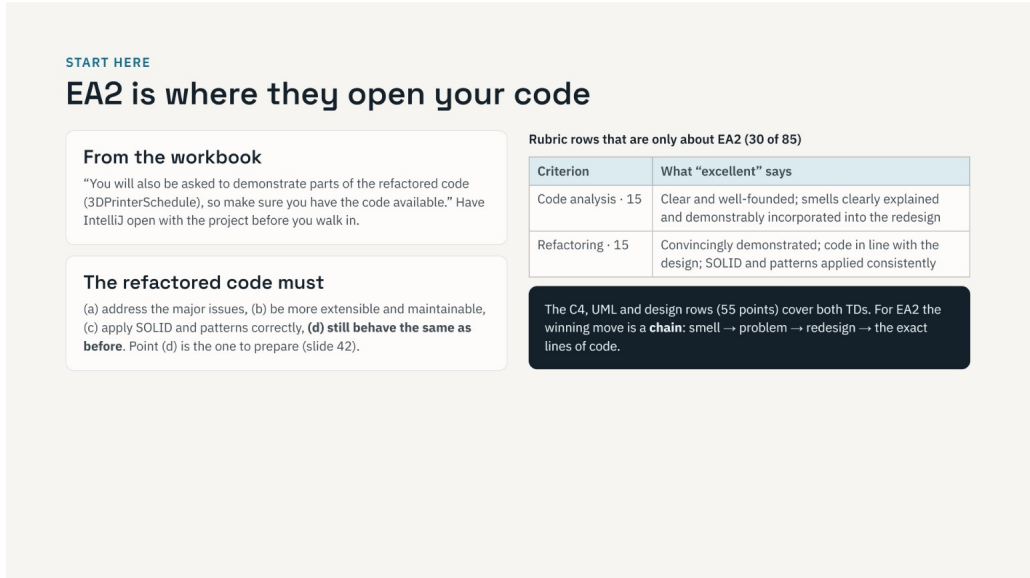
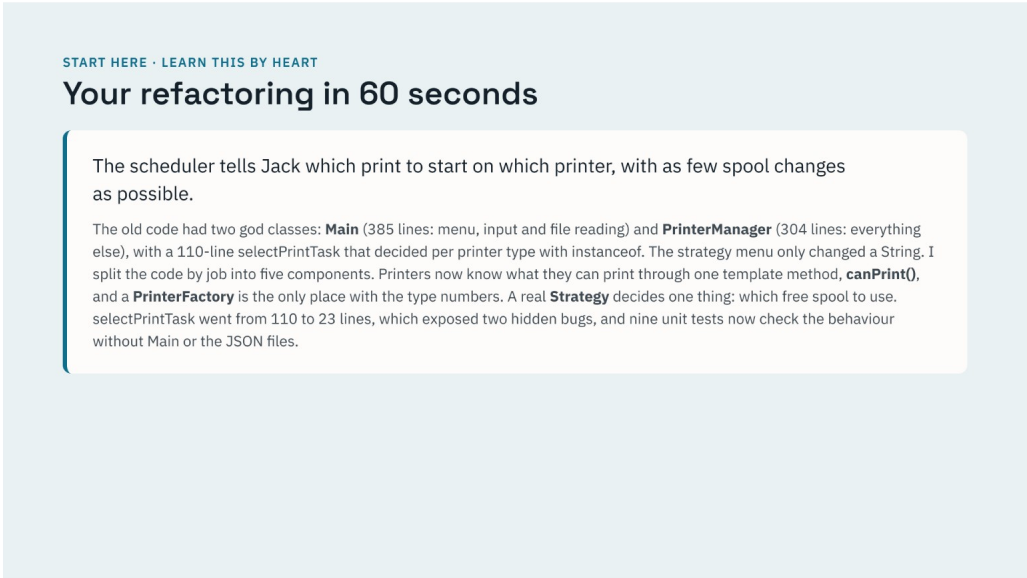


Same approach as the EA1 deck: go through it once tonight, then drill slides 41 to 48 out loud. EA2 is the part where they ask you to open your code, so slides 32 to 38 (the code, the demo script and the tests) matter most. Tonight, also open the project in IntelliJ and run the 9 tests once, so nothing surprises you tomorrow.



"Demonstrably incorporated" means: for every smell, you can point at the class or method that fixed it. Your smells table already has a "Solved by" column; this deck turns that into a demo you can walk through in the code (slide 37). For (d), have one calm sentence ready: same behaviour for every normal case, changed only where the old code crashed, lost a task, or did nothing.



Use this when they say “Tell us about EA2.” It names the two god classes, the long method, the fake strategy, the three patterns (Template Method, Simple Factory, Strategy) and the tests, so every follow-up question lands on something you prepared.

START HERE · THE MENTAL MODEL

Four problems, four fixes

1	**Two god classes, no separation.** Main mixes menu and file reading; PrinterManager does the rest.	Split by job: Console UI (Main), DataLoader, PrinterManager, Print Strategies, Domain Model.
2	**Printer types not used as types.** instanceof branches decide what a printer can print; a new type = 6 places.	Printer.canPrint() as a template method; each subclass answers its part; PrinterFactory holds the type numbers.
3	**A fake strategy.** The menu only changed a String; nothing behaved differently.	Strategy pattern: PrintStrategy.chooseSpool() with two working strategies.
4	**State in three places, one huge method.** spools, freeSpools and spool fields kept in sync by hand.	One List<Spool> per printer; selectPrintTask split into short methods, which exposed two bugs.

These are the three main problems from your presentation (slide 9 of your deck), with the spool state split out as a fourth. Any question about EA2 maps onto one of these rows. If you blank, start with: “The core problem was that printer types were not used as types.”

START HERE · NAVIGATION

Where everything is in your PDF

§	What	Page
2.1	Context diagram	4
2.2	Use case diagram	5
3.1	Container diagram, table	6
3.2	Deployment diagram	7
4.1	Old code: overview, lines	7-8
4.2	Old code: 3 class diagrams	9-12
4.3	How the old code works	12-13
5	Code smells table	14-15
6.1	Structural problems	17
6.2	Bugs, behaviour changed	17-18

§	What	Page
7.1	Layer 3 components, mapping	19-22
7.2	Domain model (printers, data)	23-25
7.3	Manager, strategies, loader, UI	26-28
7.4	SOLID and patterns	28-29
8	Refactoring per file	30
9	Testing (9 tests)	31
10.1	Decisions 1 to 5	32-33
10.2	Insights	33
code	3dprintscheduler-main/src	slide 37

Page numbers are what the PDF viewer shows (title page = 1). The table of contents inside your TD2 shows “9” for almost every section, so do not navigate by it. The two places they will most likely ask you to open: the smells table (section 5) and the decisions (section 10.1).

THE CASE · JACK'S REQUIREMENTS

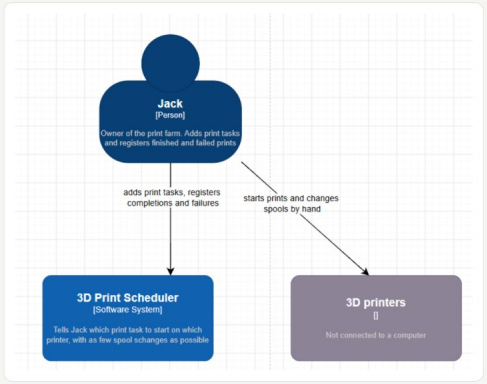
Each requirement, and where it lives now

Requirement	In the refactored code
F1 · menu with number input (U6)	Main.menu(), menuChoice(), numberInput()
F2 · printers with spools and jobs	Main.showPrinters(); Printer.toString() with the details() hook
F3, F4 · list prints; spools with length	Main.showPrints(), showSpools()
F5 · add a task: print, filament, colors	choosePrint, chooseFilamentType, chooseColors → manager.addPrintTask()
F6 · start the queue, fill every printer	PrinterManager.startInitialQueue() → selectPrintTask per printer
F7 · completion: reduce spool, pick next	registerCompletion() → finishPrintTask() + selectPrintTask()
F8 · failure: back to queue, reduce, pick next	registerPrinterFailure(): re-queue, finishPrintTask(), selectPrintTask()
F9 · choose a strategy (U10, B6 leftovers)	PrintStrategy + two strategies; Main.changePrintStrategy()
NF1-NF4 · one computer, no printer link, local files, text UI	One Java 21 console app reading JSON files; Jack handles the printers by hand

Two case facts that explain design choices. First, the printers are not connected to any computer (NF2), so there is no relation between the scheduler and the printers in your C4 diagrams. Second, “in the future Jack would like an alternative method of scheduling where small remaining spools can be used up efficiently” (B6): that is exactly your Efficient Spool Usage strategy. U1 and U2 (add printer, add print) are done by editing the JSON files, as in the original; the case says adding printers “doesn’t happen a lot”.

C4 LAYER 1 · CONTEXT · §2.1

Jack is the link to the printers



One person: Jack, owner of the print farm. He adds tasks and registers finished and failed prints.

One system: the 3D Print Scheduler. It tells Jack which task to start on which printer, with as few spool changes as possible.

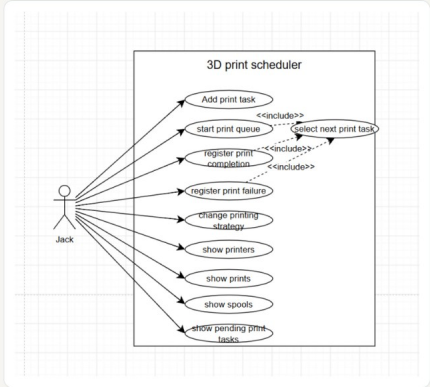
The printers are external, with no arrow to the scheduler. They are not connected to any computer (NF2). The scheduler tells Jack; Jack does it by hand.

Notation check: the printers' type label is empty ("[]") and there is no legend.

Say: "The only user is Jack. The 3D printers are an external system, but there is no relationship between them and the scheduler, because they are not connected to any computer. Jack is the link between the two: the scheduler tells him what to do and he does it by hand." If they point at the empty label: "It should say what the element is, for example [Hardware, not connected]. And a legend should be there, as in my EA1 diagrams."

C4 LAYER 1 · UML USE CASE · §2.2

What Jack does with the scheduler



Nine use cases, one per menu option: add task, start queue, register completion, register failure, change strategy, and four “show” lists.

«include» select next print task from start queue, completion and failure: in all three the program picks a task for a free printer. Dashed arrow from the base to the included case: correct.

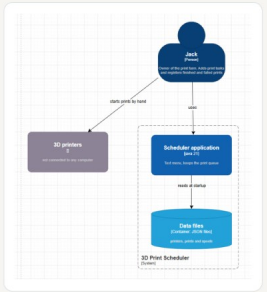
No “add printer” or “add print”: those are done in the JSON files (U1, U2), as in the original.

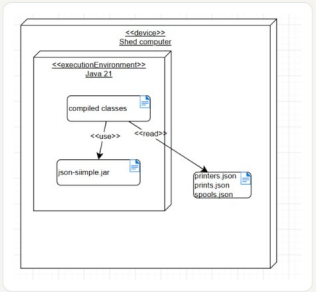
Notation check: the actor-to-use-case lines have arrowheads; in UML they are plain lines.

This diagram is better than your EA1 use case diagrams: ovals, a system boundary and a correct dashed «include». The «include» is the key point: “select next print task” is the heart of the program and three use cases share it, which is exactly why selectPrintTask is called from three places in the code (startInitialQueue, registerCompletion, registerPrinterFailure).

C4 LAYER 2 · CONTAINER AND DEPLOYMENT · §3

One app and its data files, in the shed





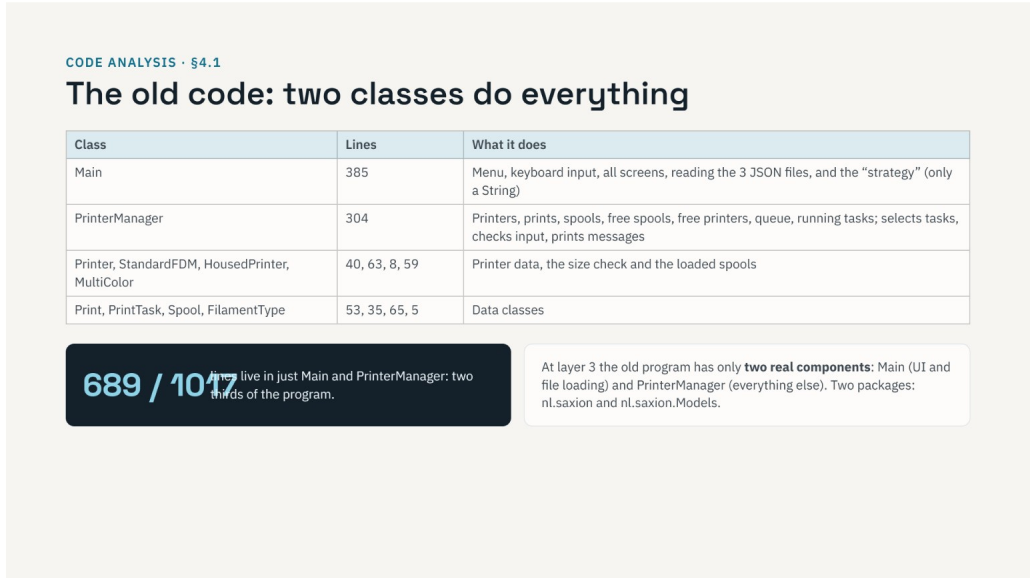
Scheduler application (Java 21 console app): menu, queue, running tasks and spool lengths in memory; picks the next task.

Data files (JSON): printers, prints, spools; read at startup.

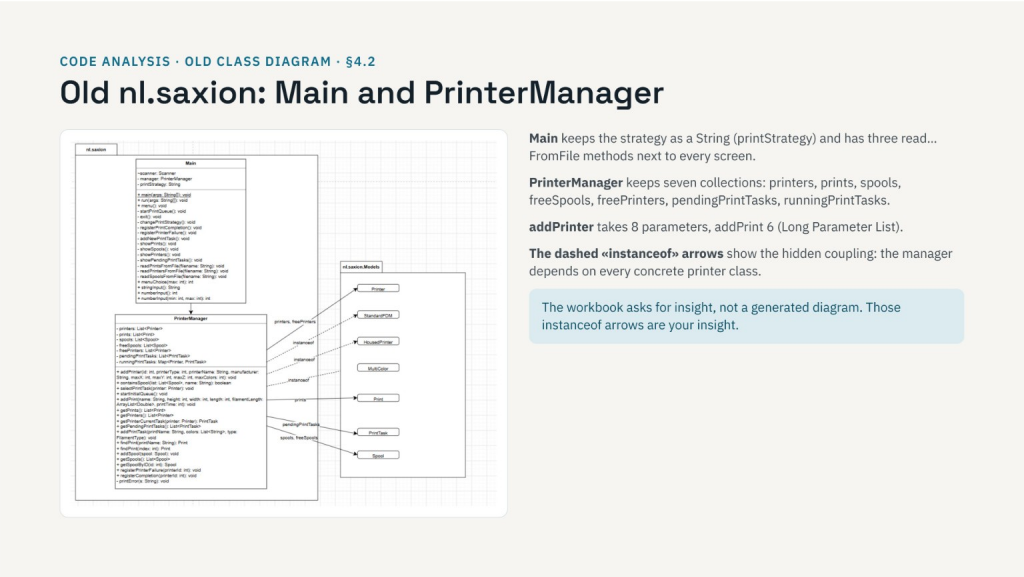
Deployment: one «device» Shed computer; Java 21 runs the classes, which use json-simple.jar and read the JSON (NF1, NF3).

Check: label “[java 21]” should be “[Container: Java 21 console app]”; “json-siimple” typo.

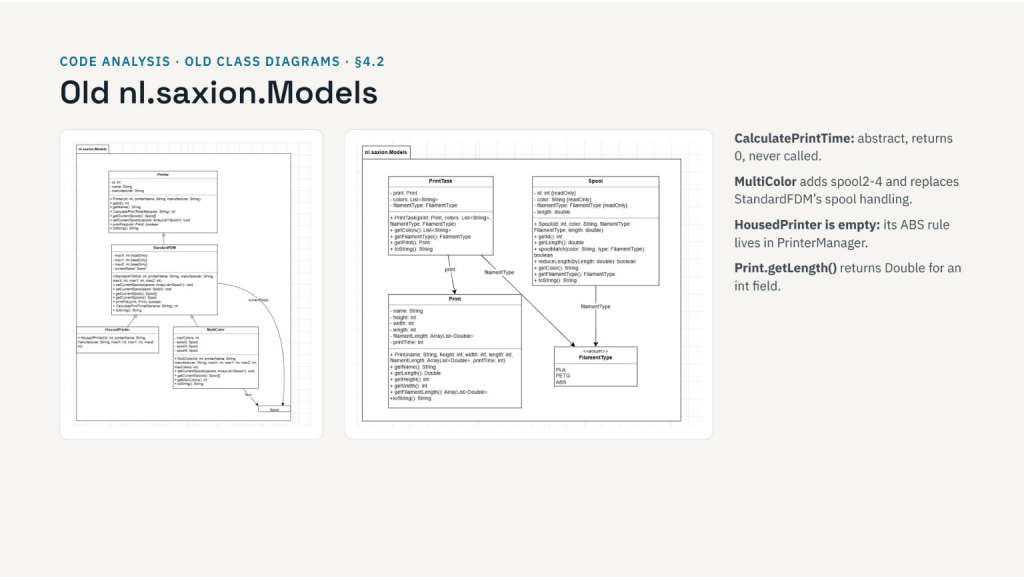
Why so small? Jack is the only user, on one small computer in his shed, and the case explicitly asks for a single-computer program with a text interface and local files (NF1 to NF4). The workbook lets you add containers when they are justified; nothing here needed one, and adding a database or server would be speculative generality, the same smell you removed from the code. If asked “where are the JSON files at runtime?”: DataLoader reads them from the classpath with getResourceAsStream, from the resources folder, just like the original read them via getResource.



The 689 of 1017 figure is computed from your own line counts (385 + 304, of 1017 lines in the ten source files); it is not written in the PDF, so say it as “about two thirds”. Opening line for the analysis: “The code has two packages. At layer 3 it has only two real components: Main does the user interface and file loading, and PrinterManager does everything else.”



A generated class diagram would not show the instanceof checks, because they are not fields or method signatures. You drew them as dependencies on purpose, to show why a new printer type means editing PrinterManager. Say that if they ask “how is this different from a generated diagram?”.



Smell names for each point: CalculatePrintTime is Speculative Generality (and wrong naming: methods start lower-case in Java). MultiColor is Refused Bequest plus Data Clumps (spool2, spool3, spool4 and new Spool[4]). HousedPrinter is a Lazy Class. The printer classes are mostly data holders, because the decisions about them sit in PrinterManager: that is the root problem you fixed with canPrint().



This is the slide to explain the core of the old program. Say: "selectPrintTask works in two passes: first it tries to keep the spool, then it changes the spool. Inside each pass there are three branches chosen with instanceof, one per printer type. So the rules of every printer type are in this one method, not in the printer classes." Fun fact you can use: the original method runs from line 37 to 149 of PrinterManager.java.

CODE ANALYSIS · SMELLS (1 OF 3) · §5

Bloaters, duplication and type checks

Smell: where	Why it hurts	Solved by
Large Class: Main	A file-format change and a menu change touch the same class	DataLoader reads the files; Main is only the console UI
Large Class, Divergent Change: PrinterManager	Storage, queue, scheduling, validation, output in one class	Rules to the printers, spool choice to strategies, reading to DataLoader
Long Method: selectPrintTask (110 lines, 6 levels)	Hard to read, impossible to test in parts	Split: selectPrintTask, chooseFreeSpools, changeSpools, startPrintTask
Duplicate Code: start block 6×, two equal branches, completion vs failure, 3 read methods	A fix in one copy is forgotten in the others	startPrintTask, finishPrintTask, readJsonArray, FilamentType.values()
Switch Statements: instanceof; type number if/else	A new printer type = changes in 6 places	Printer.canPrint(); PrinterFactory

For each row, practise the full chain: smell name, where, why it hurts, and the class or method that fixed it. That chain is what “demonstrably incorporated into the redesign” means in the rubric. Smell names follow refactoring.guru, the same source you used in the week 5 and 6 exercises.

CODE ANALYSIS · SMELLS (2 OF 3) · §5

Weak types, dead weight

Smell: where	Why it hurts	Solved by
Refused Bequest, Lazy Class: MultiColor, HousedPrinter	Hard to say what a printer can do	Spools as one list in Printer; subclasses override only what differs
Data Clumps, Primitive Obsession: spool2-4, type as int, strategy as String	Max 4 colors hard-coded; a String can't do anything	List<Spool>, PrinterFactory, PrintStrategy objects
Long Parameter List: addPrinter (8), addPrint (6)	Easy to swap two ints by mistake	The manager gets ready objects from DataLoader and PrinterFactory
Dead Code: freePrinters, getSpoolByID, containsSpool, exit(), imports	Noise, and freePrinters still had to be kept in sync	Removed
Speculative Generality: CalculatePrintTime	Every new printer class must implement it for nothing	Removed

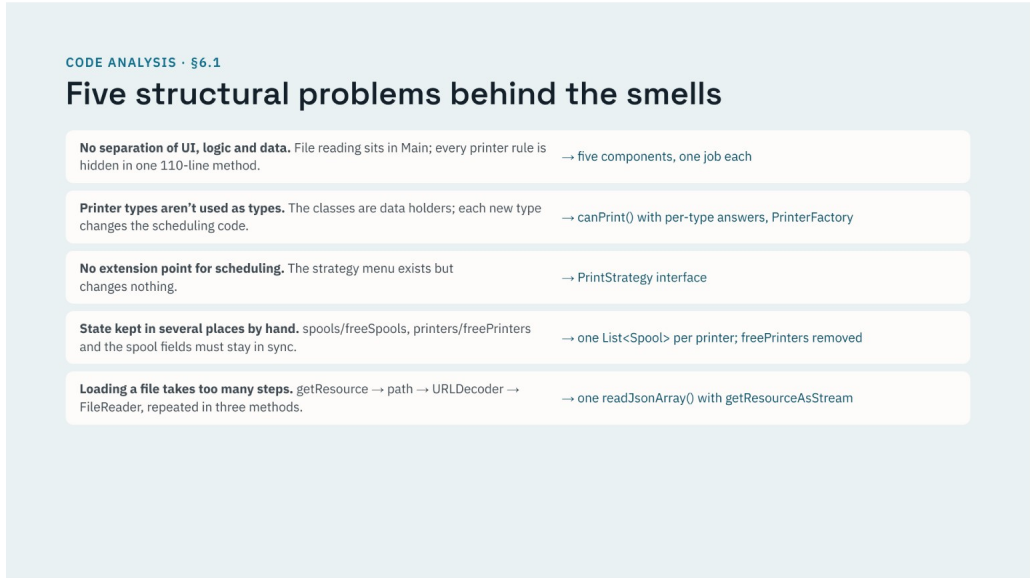
Primitive Obsession is the cleanest example of “demonstrably incorporated”: the strategy was a String in Main, now it is a PrintStrategy object the manager actually calls. The type number was an int that PrinterManager interpreted with if/else, now only PrinterFactory knows it. The four spool fields became one List<Spool>, so the maximum of 4 colors is no longer built into the class.

CODE ANALYSIS · SMELLS (3 OF 3) · §5

Misplaced logic, and one smell you kept

Smell: where	Why it hurts	Solved by
Feature Envy: PrinterManager decides what a printer can print; showPrinters edits toString() text	Logic far from the data it uses	Printer.canPrint(); the current task on its own line
Fragile inheritance: subclasses replace and split the superclass's toString() text	Lines shown twice; a superclass change breaks subclasses	Template Method: toString() calls the details() hook
Comments (fake feature): "this method only changes the name but doesn't actually work"	Jack thinks he changed the strategy	Strategy pattern with two working strategies
Divergent Change: PrinterManager writes to System.out	Another kind of UI would mean changing PrinterManager	KEPT on purpose: same console text; tests check state, not text

The kept smell is a strength if you explain it: you saw it, named it, and chose not to fix it for a reason. Reason: requirement (d) asks for the same behaviour, and the console messages are that behaviour for Jack; the tests check the state (which printer runs which task, which spool is loaded), so the printing does not get in the way of testing. Next step if they ask: PrinterManager returns results or notifies a listener, and the console UI prints them.



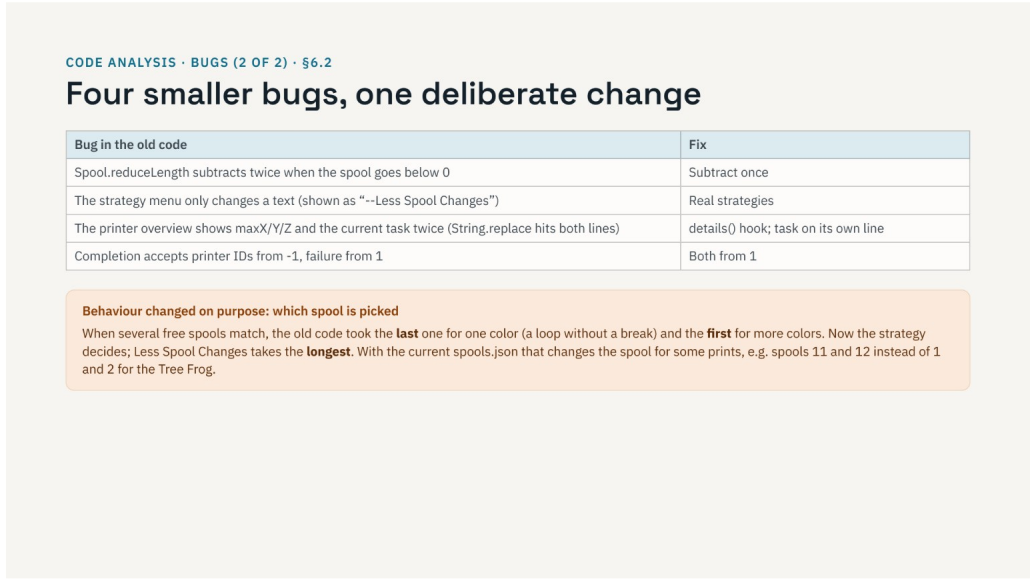
Smells are symptoms; these five are the causes. If they ask “what is the difference between a smell and a structural problem?”: a smell is a local sign in the code (a long method, a type check), a structural problem is how the program is organised (no layering, no extension point). Several smells come from one structural problem: Switch Statements, Feature Envy, Refused Bequest and Lazy Class all come from “printer types aren’t used as types”.

CODE ANALYSIS · BUGS (1 OF 2) · §6.2

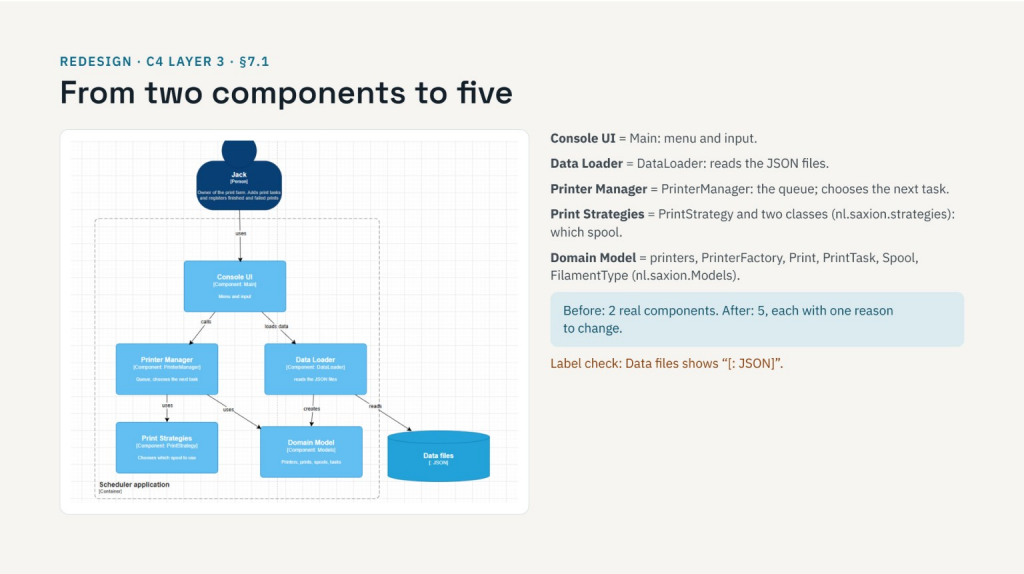
Four bugs that crash or lose work

Bug	What happened in the old code	Fix
Null spool	A printer without a spool gets its first one: freeSpools.add(getCurrentSpools()[0]) adds null. The next printer hits a NullPointerException; starting the queue with tasks for 2 printers crashes.	changeSpools only moves real spools (the list is empty at first)
Too few spools	Pass 1 for a MultiColor reads spools[i] without a null check: fewer loaded spools than colors crashes.	hasSpoolsFor first compares the counts
Busy printer	Pass 1 has no busy check: starting the queue twice replaces a running task, and that task is lost.	selectPrintTask returns at once when the printer is running
Spool in 2 printers	MultiColor keeps old spool2-4 when the new task has fewer colors, while they also went back to freeSpools.	All old spools go back first; the printer holds only the new list

The busy-printer and too-few-spools bugs are the ones you found because you split the method: after the 110 lines became four short methods, the missing null check and the missing busy check were visible. That is your third insight (“small methods make bugs visible”). If asked “how do you know these are real?”: the null spool one shows up as soon as the queue starts with tasks for two printers.



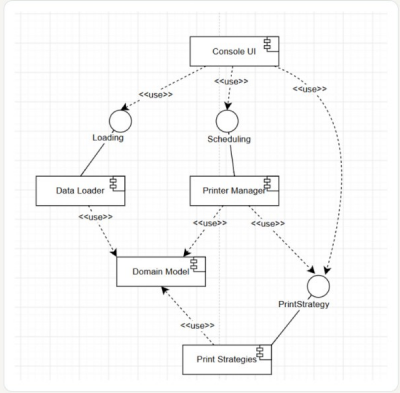
You wrote this down yourself in 6.2, which is good: a refactoring that changes behaviour must say so. Your reason for keeping bug fixes despite requirement (d): "keeping them would mean keeping crashes". Your reason for the spool choice: the old choice was accidental (a loop without a break), not a rule anyone designed, and the longest spool lasts longest before the next change, which is what "less spool changes" means. Other one-color prints affected: Red, Green and Pink PETG and Red ABS.



The workbook says layer 4 is the focus and redesigning layer 3 is optional; you did both, and the table in 7.1 maps every layer-3 component to its classes and package, which ties the two layers together (consistency, the C4 rubric row). Say: “Now every component has one job: the console UI, the data loader, the printer manager, the strategies and the domain model.”

REDESIGN · UML COMPONENT · §7.1

The interfaces between the components



Loading = DataLoader's public methods. **Scheduling** = PrinterManager's public methods. **PrintStrategy** = a real Java interface.

Console UI uses all three: it loads the data, drives the manager, and creates the strategy the manager gets. Main is the composition root: it wires everything.

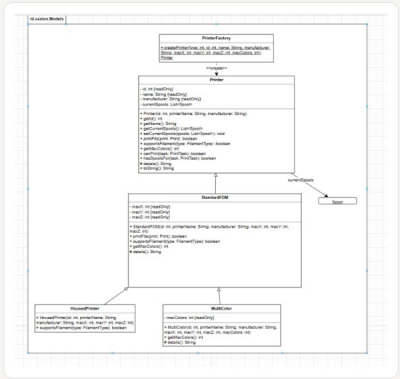
Dependencies point one way. PrinterManager knows neither DataLoader nor Main, so it can be tested alone.

Everything uses the Domain Model; the Domain Model uses nothing above it.

From your text: “The C4 diagram shows the responsibilities, the UML diagram shows the interfaces between the components.” Why does the Console UI depend on PrintStrategy? Because the strategy menu creates the strategy object and gives it to the PrinterManager. That is Dependency Inversion in practice: the manager depends on the interface, and the outermost part of the program chooses the implementation.

REDESIGN · LAYER 4 DOMAIN MODEL · §7.2

Printers now know what they can print



canPrint(task) is written once in Printer: printFits(print) and supportsFilament(type) and colors ≤ getMaxColors(). Template Method.

Each subclass answers its part. StandardFDM: size check, no ABS, 1 color. HousedPrinter: every filament. MultiColor: up to maxColors.

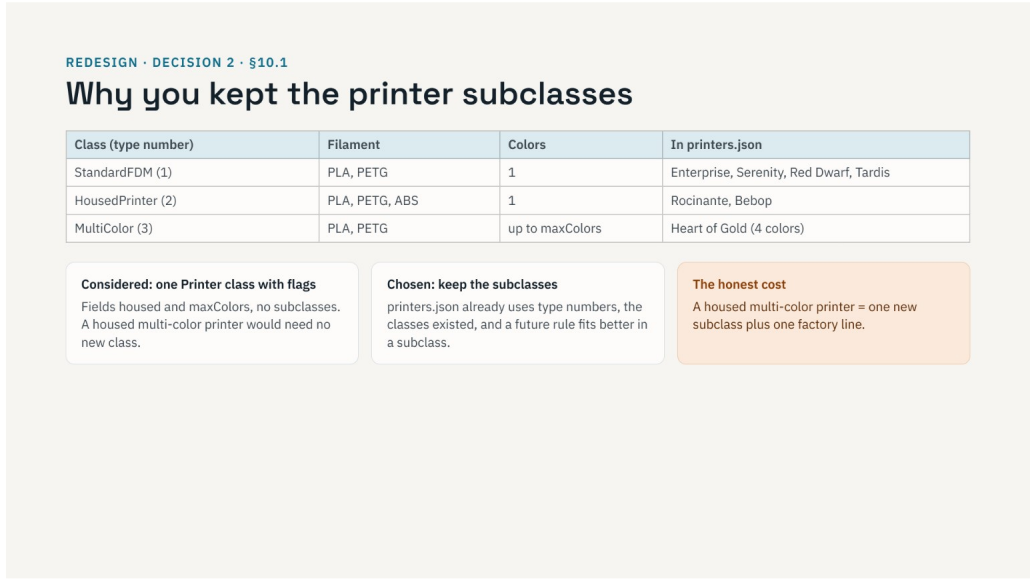
Spools: one ordered List<Spool>; the position in the list is the position in the printer.

hasSpoolsFor(task): do the loaded spools already match? That is pass 1.

details() is a protected hook (#) for toString().

PrinterFactory is the only place with the type numbers 1, 2, 3.

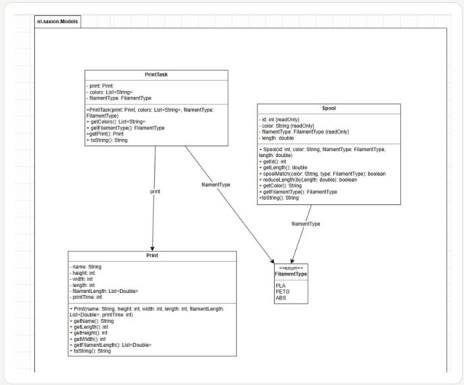
Say: “The printers now know what they can print. canPrint is written once in Printer as a template method. Each subclass only answers: does the print fit, is the filament supported, and how many colours. PrinterFactory is the only place with the type numbers.” Notation: italic method names in Printer are abstract; # is protected; the underlined method in PrinterFactory is static. An unknown type number throws an IllegalArgumentException.



Expect a challenge here, because the case itself says “a printer can easily be placed in housing” and “printers can also be upgraded to carry more colors”. That is the best argument for the flags design. Your answer: “True, and I considered it. I kept the subclasses because the data already uses type numbers and the rules really differ per type; if Jack builds a housed multi-color printer, that is one small subclass and one line in the factory, and PrinterManager does not change.” All printers also check the size: the print must fit within maxX, maxY and maxZ.

REDESIGN · LAYER 4 DATA CLASSES · §7.2

The data classes: small, careful changes



Print: List<Double> instead of ArrayList (program to the interface); getLength() returns int, like the field.

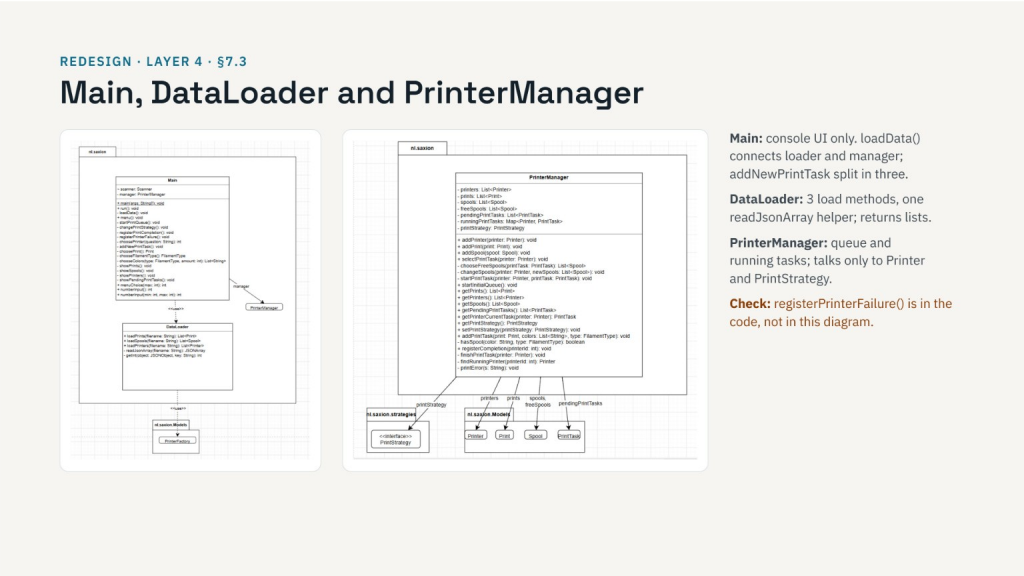
Spool: reduceLength subtracts once; spoolMatch is one line.

PrintTask: same role: a print, the chosen colors and the filament type.

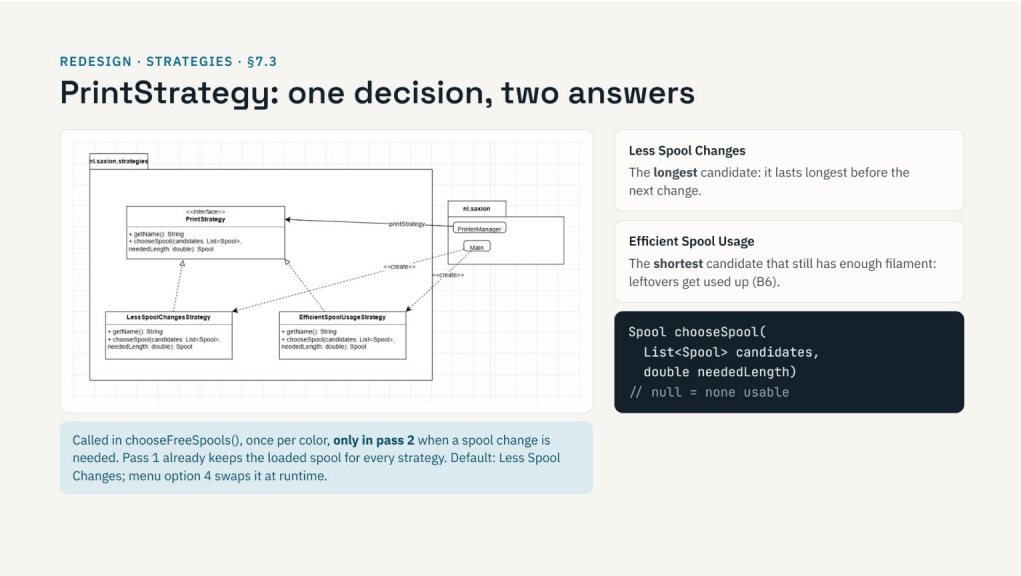
FilamentType: the menu is built from values() and files are read with valueOf(), so a new type needs no change in Main.

These classes were mostly fine. Not changing what works is also a design choice.

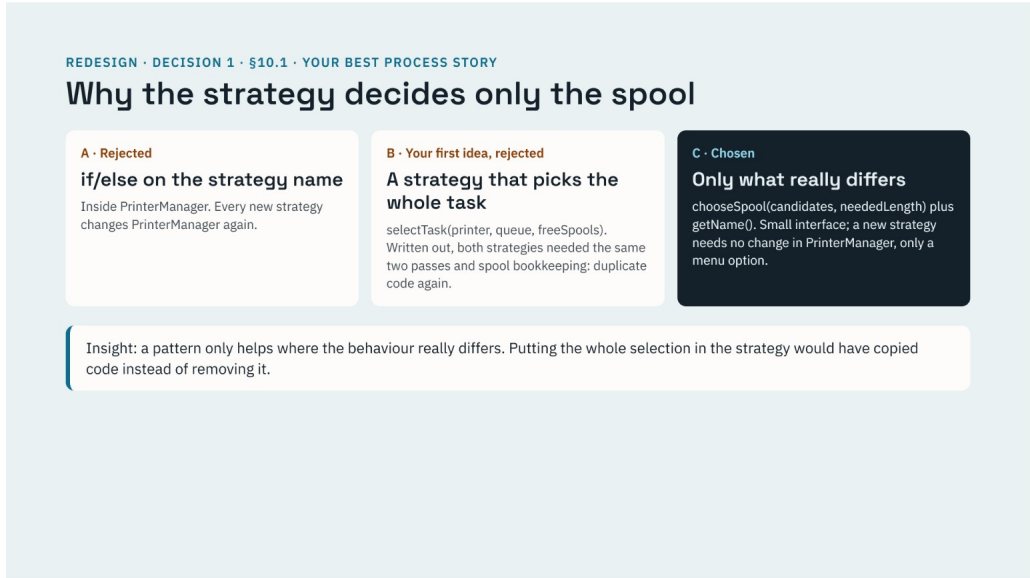
If asked why you did not change these more: they had no real smells beyond the ArrayList signatures, the Double/int mismatch and the reduceLength bug. Rewriting working code without a reason adds risk and goes against requirement (d). The duplicated filament switch in the old Main (twice) is gone because the menu now loops over FilamentType.values().



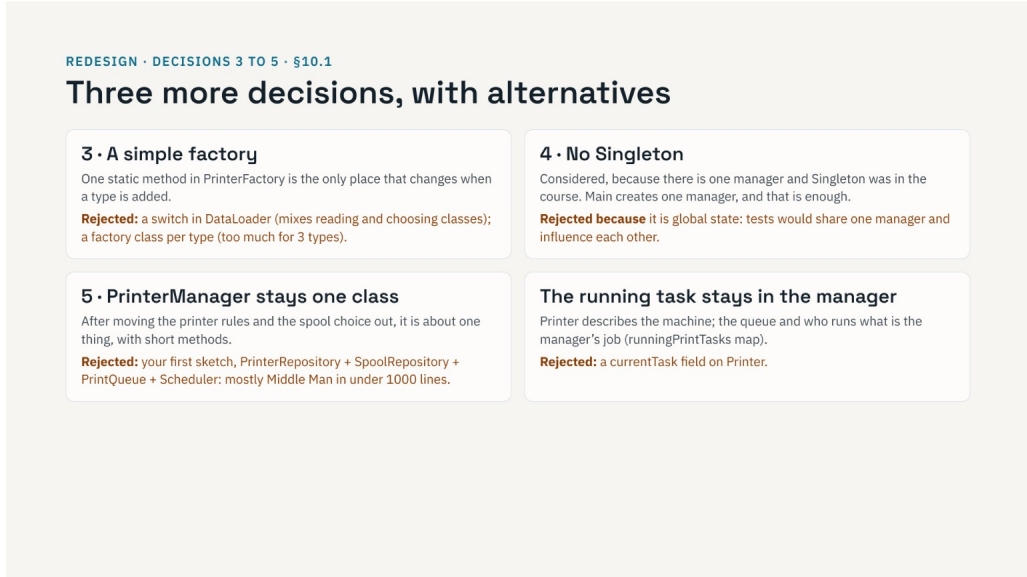
Key sentence from 7.3: “DataLoader doesn’t know PrinterManager, and Main connects the two.” That is why the tests can create a PrinterManager without files. Main’s methods: choosePrint, chooseFilamentType and chooseColors replace the long addNewPrintTask; choosePrinter is shared by completion and failure; the strategy menu creates the strategy objects. freePrinters is gone: whether a printer is free is simply “not in runningPrintTasks”.



Say: “For the third problem I used the Strategy pattern, with one decision only: which free spool to use. PrinterManager only knows the PrintStrategy interface.” Know the difference in behaviour: Efficient returns null when no candidate is long enough, so the task waits; Less Spool Changes always returns the longest, even if it is too short (the original did not check the length either). getName() is used by the menu to show the current strategy.



This is the strongest answer to “Which alternatives did you consider?” and “How did you arrive at this design?” because it shows a first idea, a test of that idea (writing out both strategies), a reason to drop it, and a better design. Say it as a story: “My first idea was a strategy that picks the whole next task. When I wrote out both strategies, they needed the same two passes, so that code would be copied into every strategy. The only thing that really differs is which free spool is taken, so that became the one decision.”



Decision 4 links to EA1: there you also rejected the Singleton, for a different reason (several battles at once). Here the reason is testability: a Singleton manager would carry state from one test to the next. Decision 5 shows you know smells work in both directions: splitting too much creates Middle Man classes that only pass calls on.

REDESIGN · SOLID · §7.4

SOLID: where each principle lives

S	**Single responsibility**	Main = console UI, DataLoader = files, PrinterManager = queue and assigning, printers = what they can print, strategies = which spool.	A new file format only touches DataLoader.
O	**Open/Closed**	New printer type: subclass + one factory line. New strategy: class + one menu option. New filament: one enum value.	PrinterManager changes for none of these.
L	**Liskov substitution**	PrinterManager only uses Printer. HousedPrinter only allows more filament, MultiColor more colors.	Every subclass works everywhere.
I	**Interface segregation**	PrintStrategy has only the two methods the manager and the menu need.	No strategy implements what it doesn't use.
D	**Dependency inversion**	PrinterManager depends on the PrintStrategy interface, not on the concrete strategies; Main chooses which one is used.	Strategies swap at runtime.

Two honest footnotes, in case they dig. Open/Closed: a new strategy still needs a menu option in Main, but PrinterManager stays closed. Dependency inversion: PrinterManager does create its default strategy itself (new LessSpoolChangesStrategy() as the field's initial value), so it imports one concrete class. Cleaner would be Main passing the strategy in through the constructor. Say that before they find it.

REDESIGN · PATTERNS · §7.4

Three patterns, and one you rejected

Pattern	Where	Why	Precisely
Strategy	PrintStrategy, LessSpoolChanges, EfficientSpoolUsage	Jack switches spool choice while running; new strategies later	Interchangeable algorithms behind one interface, chosen at runtime
Simple Factory	PrinterFactory.createPrinter()	One place turns the type number into a class	An idiom, not a GoF pattern; Factory Method would need creator subclasses
Template Method	Printer.canPrint(); toString() with details()	Steps fixed in Printer; subclasses fill in their part	canPrint calls abstract steps; details() is a hook with a default
Singleton	Considered for PrinterManager	There is only one manager	REJECTED: global state, tests would share it

Template Method in one sentence: the base class fixes the order of the steps, and subclasses only fill in steps. canPrint() is the classic form (it calls three abstract operations: printFits, supportsFilament, getMaxColors); toString() is the hook form (details() returns "" by default and StandardFDM and MultiColor add lines). It replaces two smells: the instanceof checks and the string replace/split in the old toString(). If they ask “why not Factory Method?”: Factory Method lets subclasses of a creator decide which object to make; here one static method with three cases is enough, and you rejected a factory class per type as too much for three types.

REFACTORING · §8

What changed, file by file

File	Change
Main	Only the console UI: loading moved out, addNewPrintTask split in three, shared choosePrinter, the strategy menu creates real strategies
DataLoader (new)	Reads the 3 JSON files with one helper, returns lists, uses getResourceAsStream
PrinterManager	Short selectPrintTask with helpers, no instanceof, shared finishPrintTask, add methods take objects, dead code removed
strategies (new package)	PrintStrategy and two implementations
Printer	Spools as a list, canPrint, hasSpoolsFor, details() hook; CalculatePrintTime removed
StandardFDM, HousedPrinter, MultiColor	Only override what is different
PrinterFactory (new)	Type number to printer class
Spool, Print, PrintTask	reduceLength fixed, spoolMatch shorter; List instead of ArrayList; getLength() returns int

Line counts after the refactoring: PrinterManager 197 (was 304), Main 234 (was 385), DataLoader 72, Printer 84, StandardFDM 36, HousedPrinter 14, MultiColor 21 (was 59), PrinterFactory 16, the strategies 13 + 24 + 24. Total roughly the same size, but spread over classes that each have one job. That is a good answer to “did the refactoring make the code bigger?”.



The comments “pass 1” and “pass 2” are shortened here; in your file they read “First we look if there’s a task that matches the current spools on the printer” and “If we didn’t find a print for the current spools we search for a print with the free spools”. Indentation is 2 spaces here to fit the slide. Say: “Pass one keeps the loaded spools, pass two changes spools through the strategy. There is no instanceof any more. Splitting the method also showed a bug: a busy printer could lose its task. That check is now the first if.” The nesting count after (3) is my count of the deepest block in this method, not a number from your PDF.

CODE · MODELS/PRINTER.JAVA LINES 40 AND 47 · SUBCLASSES

canPrint(): the template method

```java
// Printer (abstract)
public boolean canPrint(PrintTask task) {
  return printFits(task.getPrint())
    && supportsFilament(task.getFilamentType())
    && task.getColors().size() ⩽ getMaxColors();
}
public boolean hasSpoolsFor(PrintTask task) {
  List<String> colors = task.getColors();
  if (currentSpools.size() < colors.size()) {
    return false;
  }
  for (int i = 0; i < colors.size(); i++) {
    if (!currentSpools.get(i).spoolMatch(
        colors.get(i), task.getFilamentType())) {
      return false;
    }
  }
  return true;
}
```

```java
// StandardFDM
public boolean printFits(Print print) {
  return print.getHeight() ⩽ maxZ
    && print.getWidth() ⩽ maxX
    && print.getLength() ⩽ maxY;
}
supportsFilament(type): type ≠ ABS
getMaxColors(): 1

// HousedPrinter extends StandardFDM
supportsFilament(type): true

// MultiColor extends StandardFDM
getMaxColors(): maxColors

// shorthand, not literal code
```

The left card is your real code (2-space indents). The right card shows printFits in full and the other overrides in shorthand. Say: “canPrint in Printer: the steps are fixed and the subclasses fill in their part.” The orange lines in hasSpoolsFor are the fix for the old crash: compare the counts first, so a printer with fewer loaded spools than the task has colors never reads a missing spool. Notice MultiColor inherits supportsFilament from StandardFDM, so it cannot print ABS, exactly as in the original.

CODE · PRINTERMANAGER.JAVA LINES 75 AND 172

Spool bookkeeping in one place

```
private void changeSpools(Printer printer,
        List<Spool> newSpools) {
  freeSpools.addAll(printer.getCurrentSpools());
  freeSpools.removeAll(newSpools);
  printer.setCurrentSpools(newSpools);
  // + a “Spool change” message per spool
}

private void finishPrintTask(Printer printer) {
  PrintTask task = runningPrintTasks.get(printer);
  runningPrintTasks.remove(printer);
  // + “Task … removed from printer …”
  List<Spool> currentSpools = printer.getCurrentSpools();
  for (int i = 0; i < currentSpools.size()
      && i < task.getColors().size(); i++) {
    currentSpools.get(i).reduceLength(
        task.getPrint().getFilamentLength().get(i));
  }
}
```

addAll of a list: empty at first, so no null spool goes into freeSpools (bug 1).

All old spools go back first, then the printer gets only the new list: no spool in two printers (bug 4).

One finishPrintTask for completion and failure (was duplicated). A failed print also used filament.

Failure = re-queue the task, finishPrintTask, selectPrintTask (line 162).

registerCompletion (line 154) and registerPrinterFailure (line 162) now differ by exactly one line: the failure version first adds the running task back to the pending list. Both then call finishPrintTask and selectPrintTask. The failed task goes to the end of the queue, the same as in the original code. In the test failedTaskGoesBackToTheQueueAndStillUsesFilament, it is the only task, so the printer starts it again with the same spool.

CODE · STRATEGIES/ · MAIN.JAVA LINE 80 · PRINTERMANAGER.JAVA LINE 56

The Strategy, end to end

```
public interface PrintStrategy {
  String getName();
  // null when no candidate can be used
  Spool chooseSpool(List<Spool> candidates,
           double neededLength);
}

// LessSpoolChanges: the longest
if (chosen == null
    || spool.getLength() > chosen.getLength())
  chosen = spool;

// EfficientSpoolUsage: shortest that fits
if (spool.getLength() ≥ neededLength
    && (chosen == null
    || spool.getLength() < chosen.getLength()))
  chosen = spool;
```

```
// Main.changePrintStrategy()
if (strategyChoice == 1) {
  manager.setPrintStrategy(
      new LessSpoolChangesStrategy());
} else {
  manager.setPrintStrategy(
      new EfficientSpoolUsageStrategy());
}

// PrinterManager.chooseFreeSpools()
// once per color of the task:
Spool chosenSpool = printStrategy
    .chooseSpool(candidates, length(i));
if (chosenSpool == null) {
  return null;
}
```

Walk it left to right in the demo: the interface (one decision), the two answers, the menu that creates the objects, and the single place the manager calls it. “length(i)” is shorthand for printTask.getPrint().getFilamentLength().get(i). Candidates are the free spools that match the color and filament type and are not already chosen for another color of the same task. If any color gets null, chooseFreeSpools returns null and the task is skipped for this printer.

CODE · DATALOADER.JAVA LINE 55 · MODELS/PRINTERFACTORY.JAVA · MAIN.JAVA LINE 46

Loading the data, building the printers

```
private JSONArray readJsonArray(String file) {
  InputStream stream = getClass()
      .getResourceAsStream("/" + file);
  if (stream == null) {
    System.err.println("Warning: ...");
    return new JSONArray();
  }
  try (Reader reader = new InputStreamReader(
      stream, StandardCharsets.UTF_8)) {
    return (JSONArray) new JSONParser()
        .parse(reader);
  } catch (IOException | ParseException e) {
    e.printStackTrace();
    return new JSONArray();
  }
}
```

```
// PrinterFactory (condensed)
static Printer createPrinter(int type, ...)
  type 1 → new StandardFDM(...)
  type 2 → new HousedPrinter(...)
  type 3 → new MultiColor(..., maxColors)
  else   → throw new
           IllegalArgumentException(...)

// Main.loadData(): Main connects
// the loader and the manager
for (Printer printer :
    loader.loadPrinters("printers.json")) {
  manager.addPrinter(printer);
}
```

The left card is real code (the parameter is called filename in your file). It replaces three copies of getResource → file path → URLDecoder → FileReader. The try-with-resources closes the reader; a missing file gives a warning and an empty list instead of a crash. The right card condenses PrinterFactory’s if/else-if chain: in the file it is a normal if (type == 1) … else if (type == 2) … chain. That is fine there: one switch on the type number in ONE place is the point of a factory; the smell was the same decision spread over PrinterManager.

LIVE DEMO SCRIPT · PATHS UNDER SRC/MAIN/JAVA/NL/SAXION

If they say “show us the code”

#	Open	Say
1	PrinterManager.java : 31	“selectPrintTask: 110 lines became 23. Busy check first, then two passes, no instanceof.”
2	Models/Printer.java : 40, 47	“canPrint is the template method; hasSpoolsFor is pass 1.” Then open StandardFDM, HousedPrinter, MultiColor.
3	strategies/PrintStrategy.java	“One decision: which free spool.” Then both strategies, then Main.java : 80 creating them.
4	PrinterManager.java : 56, 75	“chooseFreeSpools calls the strategy; changeSpools fixed the null spool and the spool in two printers.”
5	Models/PrinterFactory.java, DataLoader.java : 55	“The only place with type numbers; one helper reads all three files.”
6	tests/nl/saxion/PrinterManagerTest.java	Run all 9 tests: green. “No Main and no JSON needed, except the truth table.”

Step 7, if they want to see it run: start Main, then menu 1 (add Octopus, PLA, a color), 5 (start queue), 7 (show printers: the task and the spool), 4 (the strategy menu now shows the current strategy’s name). Practise this once tonight with IntelliJ open, using Ctrl+N (or Cmd+O on a Mac) to open a class by name and Ctrl+G to jump to a line. If they ask for something specific, jump straight there instead of following the order. Line numbers are from the version in your submission folder (Downloads/RokasPazera551495/.../exam2/3dprintscheduler-main). If your IntelliJ project is a different copy, check the line numbers once.

TESTING · §9 · TESTS/NL/SAXION/PRINTERMANAGERTEST.JAVA

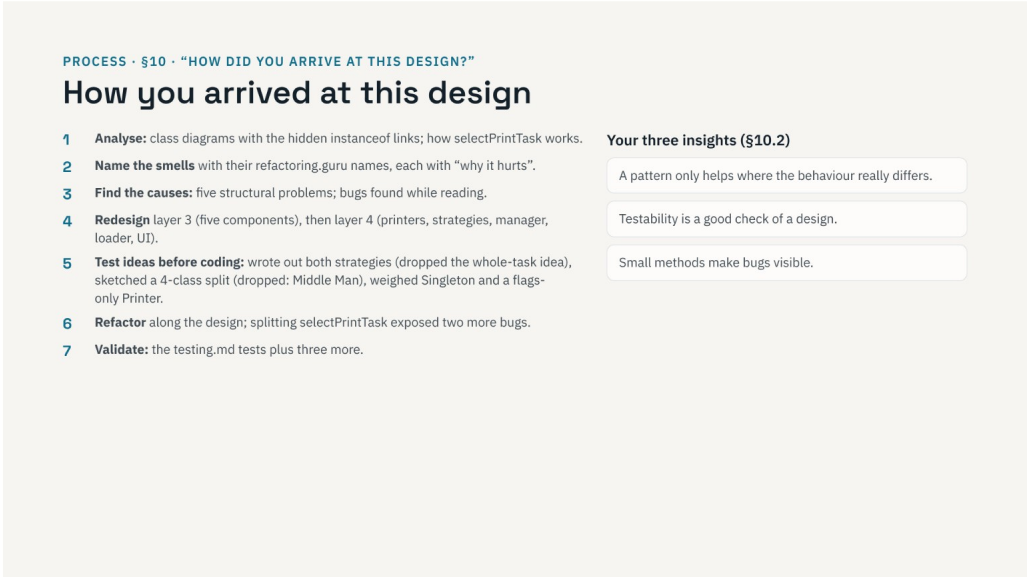
Nine tests, all green

Test (first six from testing.md)	What it proves
taskGoesToPrinterThatAlreadyHasTheMatchingSpool	Same spool kept, no spool change
spoolChangeIsInitiatedWhenThePrinterHasNoMatchingSpool	The red spool is placed for a red task
taskStaysInQueueWhenNoPrinterCanPrintIt	Moon Lamp (258 mm) stays in the queue
readyPrinterSelectsTaskThatFitsAndUsesTheSameSpool	Skips a too-big task and an other-color task
startingTheQueueFillsAllPrinters	3 printers get 3 tasks, ABS on Bebop
blueRedBlueSelectsTheThirdTaskWhenThePrinterIsReady	The third (blue) task wins over red
failedTaskGoesBackToTheQueueAndStillUsesFilament	Restarted; spool reduced by 2.86
strategiesChooseTheRightSpool	Longest (100) vs shortest that fits (5, not 1)
printersMatchTheTruthTable	canPrint for 9 prints × 7 printers, PLA and ABS

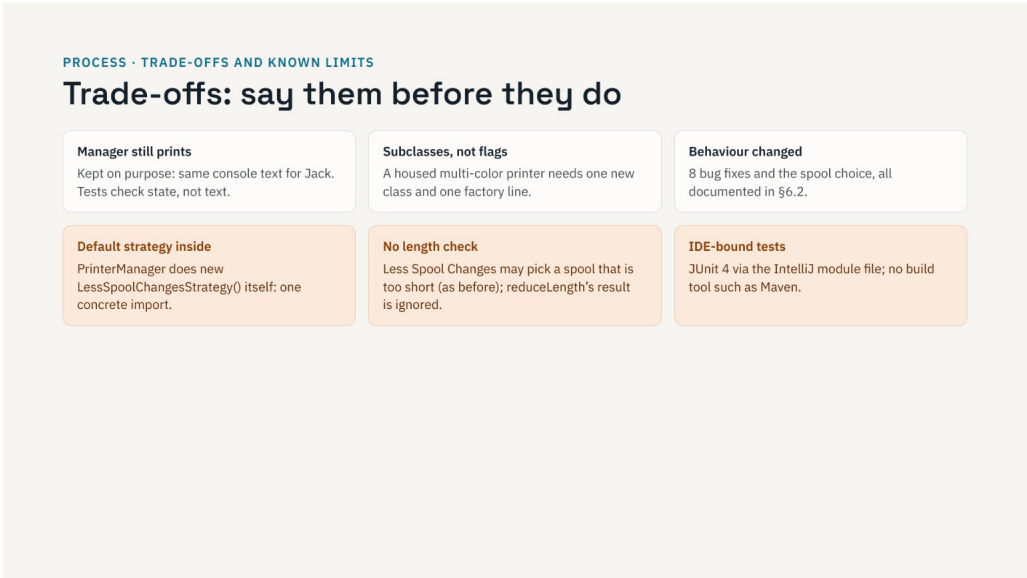
State, not text: each test builds a small situation and checks which task runs where and which spool is loaded.

Serenity: testing.md says it prints ABS; printers.json says type 1. The test follows the data.

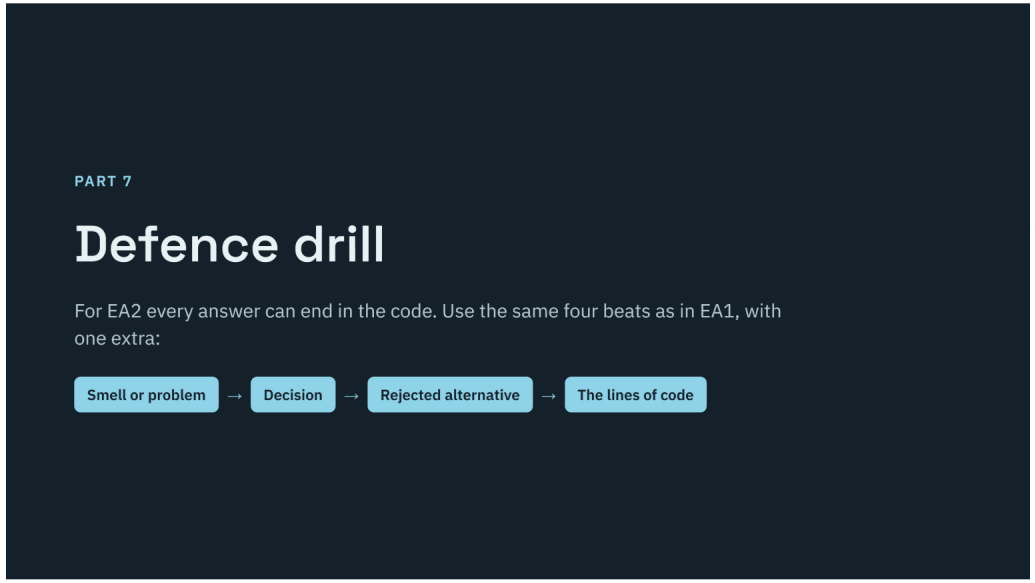
Why this was impossible before: PrinterManager was only reachable through Main and the JSON files, and printers could only be made through addPrinter with a type number. Now a test does new PrinterManager(), new StandardFDM(...), addSpool(...), addPrintTask(...), then checks getPrinterCurrentTask(). The tests are JUnit 4 (@Before, @Test) in the tests folder, which your .iml marks as the test source root. Tonight: right-click the tests folder in IntelliJ, Run ‘All Tests’, and check all 9 are green. Also know the truth-table difference: in testing.md Serenity can print ABS, but printers.json gives Serenity type 1 (StandardFDM, no ABS). You kept the data unchanged and made the test follow it; the same goes for the row order, which follows prints.json.



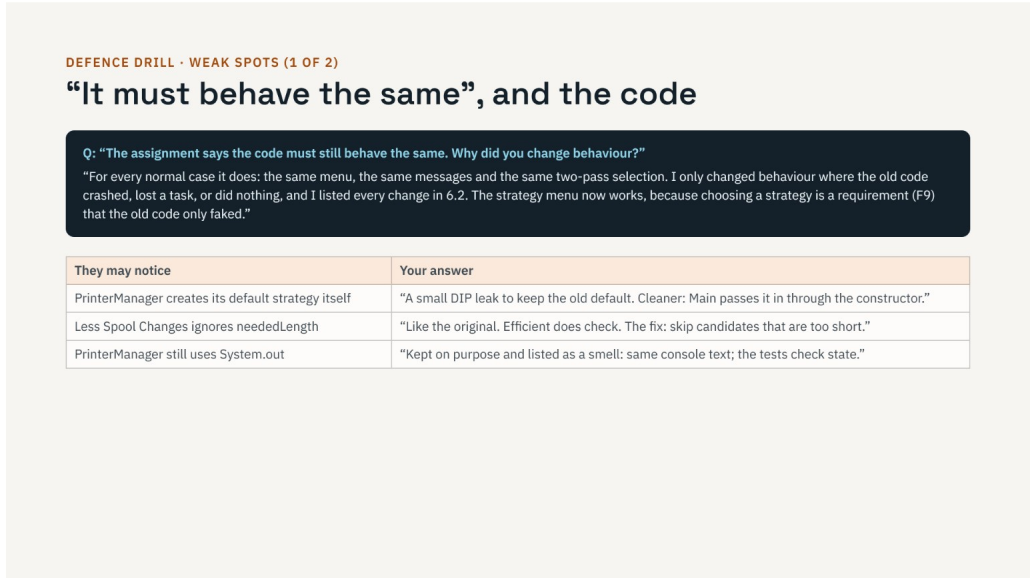
This order follows the structure of your TD2 and the decisions it describes; the exact days are not in the document, so tell it as an order, not as dates. The three insights are also the takeaways of your presentation’s last slide (“design from the interface outward, use a pattern only where the behaviour differs, use testability as a check”), so they connect both assignments.



The top row are choices you made and documented. The bottom row are limits I found in the code that are not in your PDF; name them yourself if the conversation goes there, with the fix: pass the strategy in through the constructor; skip candidates shorter than neededLength (or warn Jack); add a small Maven or Gradle build so the tests run anywhere. A spool length going below 0 does show up in “Show spools”, which at least tells Jack which spools to order (B4).



Example. Q: “Why did you use the Strategy pattern?” Problem: the strategy menu only changed a String, so nothing behaved differently, and Jack wants a way to use up leftover spools (B6). Decision: a PrintStrategy interface with one decision, chooseSpool, and two implementations that Main creates. Rejected: a strategy that picks the whole task, which needed the same two passes in every strategy. Code: open PrintStrategy.java, then chooseFreeSpools in PrinterManager line 56.



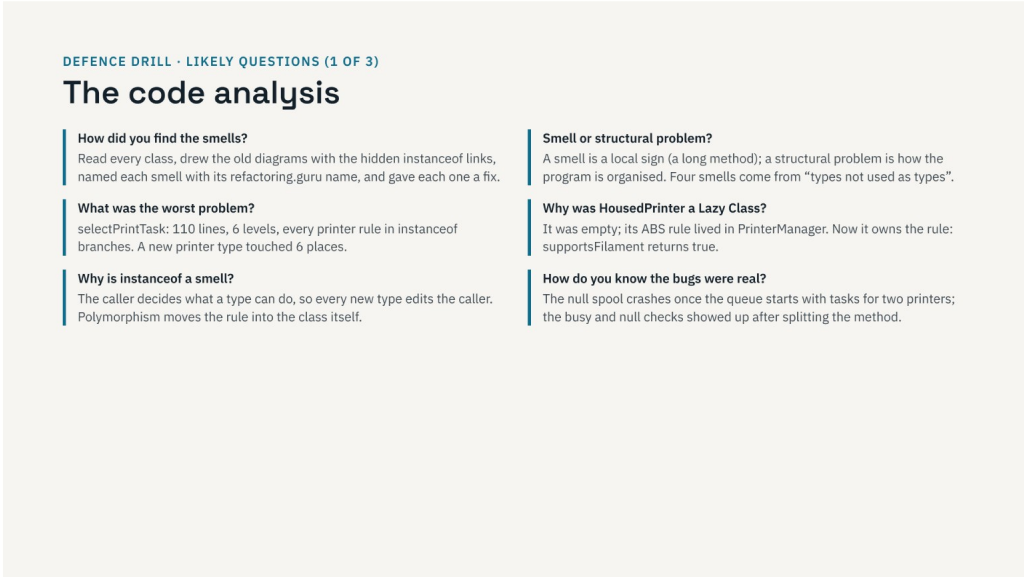
The first answer is the most important one of EA2; learn it word for word. The rows are things an examiner might spot while you have the code open. The first two are not in your PDF, so they would be new to the examiner too; naming the fix immediately shows you understand the design rather than defending it blindly.

DEFENCE DRILL · WEAK SPOTS (2 OF 2)

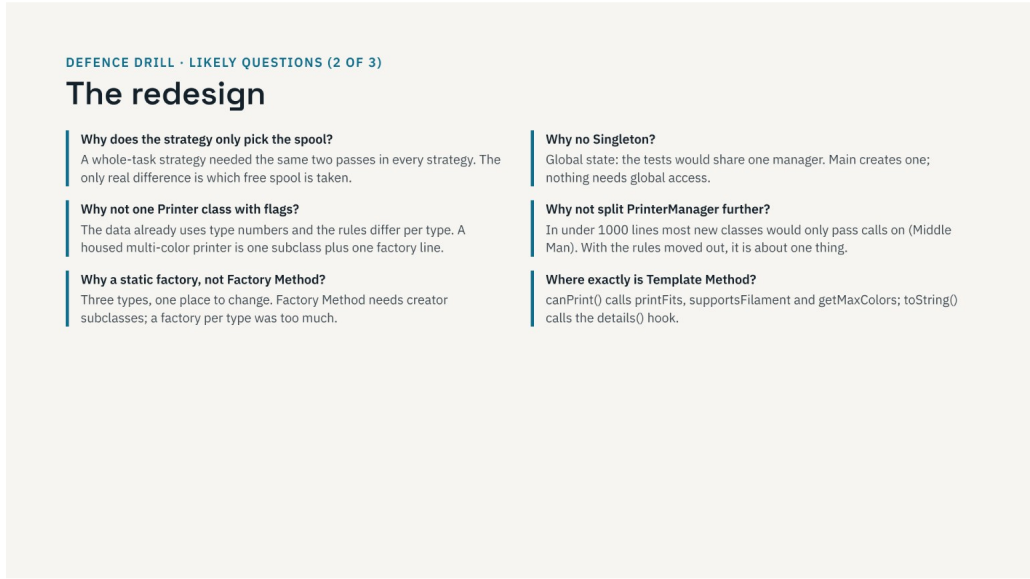
Slips in the document, and what to say

Where	What they may notice	Your answer
§2.2	Use case lines have arrowheads	“In UML the actor-to-use-case association is a plain line.”
§2.1, 3.1, 7.1	C4 labels “[]”, “[java 21]”, “[: JSON]”; no legend	“Each element should show its type and technology, with a legend, as in my EA1 diagrams.”
§3.2	“json-siimple.jar”; JSON files outside the Java box	“A typo. The files are read from the classpath, so they belong with the classes.”
§7.3	registerPrinterFailure() missing in the PrinterManager diagram	“It is in the code, line 162; the diagram misses it.”
§9	Truth-table test differs from testing.md (Serenity, ABS)	“printers.json says Serenity is type 1, so I followed the data.”
Contents	Page numbers show “9” for most sections	Navigate by section number (slide 5).

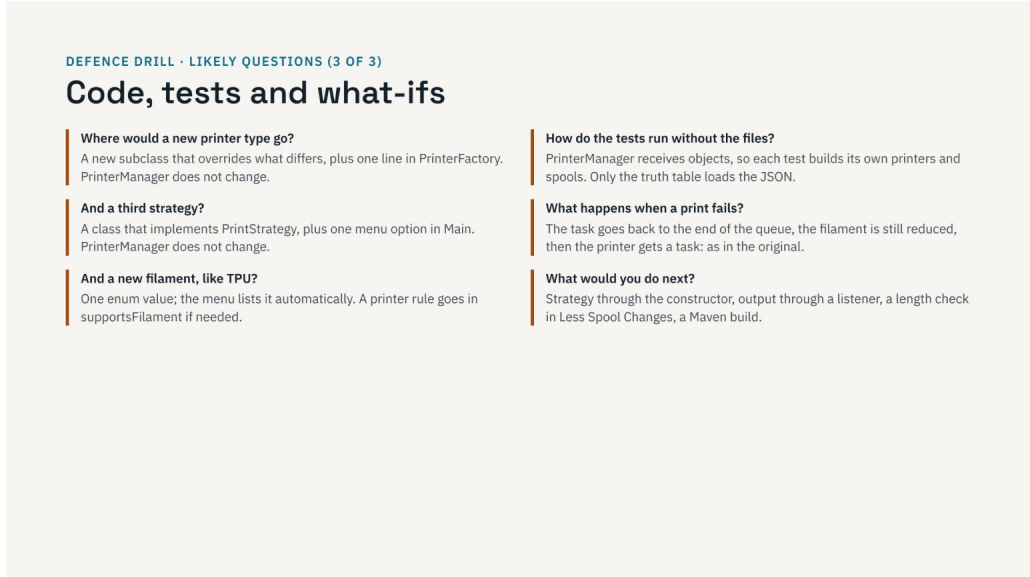
Same tone as in EA1: “Good catch, that should be X.” Then move on. None of these touch the design itself. If they push on notation in general, point out that your EA2 use case diagram uses ovals, a boundary and a correct dashed «include», and that the C4 layers line up: the five layer-3 components map one-to-one to the classes and packages in the table of 7.1.



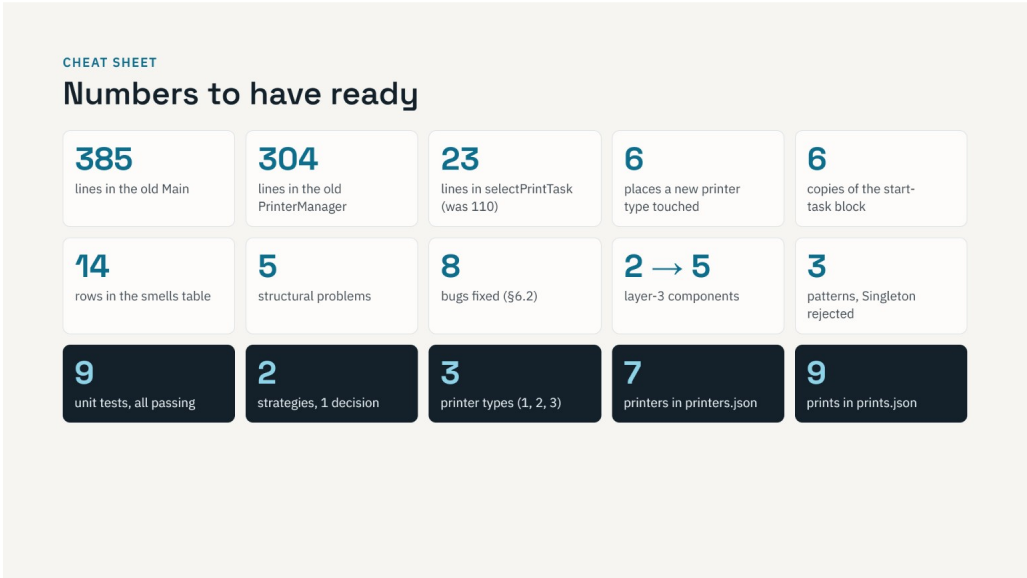
The four smells from “types not used as types” are Switch Statements (instanceof), Feature Envy (PrinterManager deciding for the printer), Refused Bequest (MultiColor) and Lazy Class (HousedPrinter). Being able to group smells under a cause is what makes an analysis “well-founded” in the rubric’s words.



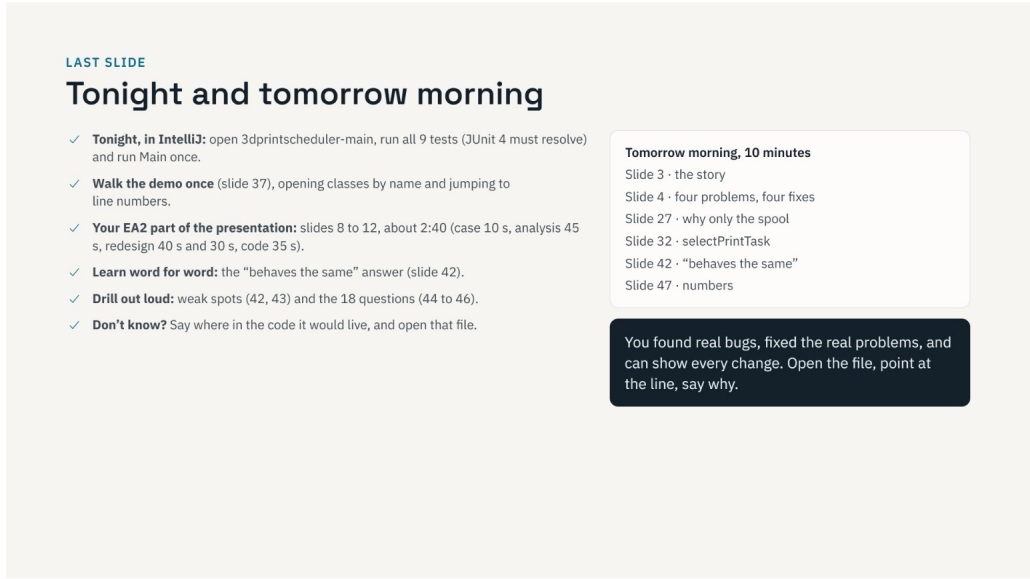
All six answers are your own decisions from section 10.1, compressed. For each one, the rejected alternative is the strongest part; say it. If they ask about Liskov: PrinterManager only uses Printer, and every subclass can stand in for it: HousedPrinter only allows more filaments, MultiColor more colors, and none of them throws or changes what canPrint means.



For the three what-ifs, offer to show it: “Shall I open PrinterFactory?” That turns a question into a demo, which is what the refactoring row of the rubric rewards (“convincingly demonstrated”). If they ask what “Open/Closed” means here, the first two answers are the proof: new behaviour by adding classes, not by editing PrinterManager.



The five components: Console UI, Data Loader, Printer Manager, Print Strategies, Domain Model. The three patterns: Strategy, Simple Factory, Template Method. The five structural problems: no separation; types not used as types; no extension point for scheduling; state in several places; loading takes too many steps. The 7 printers: Enterprise, Serenity, Red Dwarf, Tardis (standard), Rocinante, Bebop (housed), Heart of Gold (multi-color, 4 colors).



Together with the EA1 deck this covers the whole assessment. If the presentation runs long, your talking script already lists safe cuts (slide 10's last sentence, slide 11's Singleton sentence); keep those as answers for the questions instead.